

Machine Learning

Lectures 1 to 4 in One Continuous Read

Every concept, its motivation and its formula, in the order the ideas depend on each other.

Based on the lecture notes and slides of Lectures 1–4 (Prof. Mounir Ghogho, UM6P).

How to use this document

Read it front to back once; the sections build on each other and the “thread” boxes at the end of each part tell you what to carry forward. Formulas are in blue boxes, the reason a concept exists is in yellow boxes, and the classic confusions are in red boxes. The last two pages are a one-glance sheet of every formula and every “ X versus Y ” distinction in the course.

Contents

1	What does it mean to learn? (Lecture 1)	2
1.1	Learning, abilities, and the two ways to give them to a machine	2
1.2	Three forms of experience, three learning settings	2
1.3	Supervised learning in detail	3
1.4	What is presented to the model: features and representations	3
1.5	Four ingredients, and the challenge of generalisation	4
2	From representations to decisions (Lecture 2)	4
2.1	The input as a vector	4
2.2	Decision making when the distribution is known	4
2.3	From the known distribution to finite data	5
2.4	First strategy: estimate locally with nearest neighbours	6
2.5	Second strategy: parameterised models	6
2.6	Learning the parameters: empirical risk minimisation	7
2.7	Evaluating a learned model: three subsets	8
3	From fitting to generalisation (Lecture 3)	9
3.1	Two risks and the generalisation gap	9
3.2	Model capacity, underfitting, overfitting	9
3.3	Bias and variance	10
3.4	Controlling complexity	10
3.5	Feature selection and model selection	11
3.6	Reliable evaluation: splits, metrics, leakage	11
4	Discriminative and generative classification (Lecture 4)	12
4.1	The decision is easy; the posterior is the problem	12
4.2	The discriminative route	12
4.3	The generative route	13

4.4	Gaussian class models: LDA and QDA	14
4.5	Naive Bayes	15
4.6	Comparing the two routes	16
4.7	Imbalance, prior shift, calibration	16
4.8	Unfamiliar inputs and combined systems	16
	One-glance sheet: formulas and distinctions	18

1 What does it mean to learn? (Lecture 1)

1.1 Learning, abilities, and the two ways to give them to a machine

Learning is the process through which **experience** changes a learner and develops what it can do. The lecture's first picture is

$$\text{Capacity to learn} + \text{Experience} \xrightarrow{\text{learning}} \text{Acquired abilities.}$$

The abilities we care about fall into four overlapping activities: **recognise** (what is it?), **predict** (what may happen?), **reason** (what follows?), **act** (what should I do?). The driving question of AI is how to give machines these abilities.

First idea: encode human knowledge. Humans learn first, then transfer the result as explicit rules: Human knowledge $\rightarrow$ rules or procedures $\rightarrow$ computer. This is still right whenever a reliable procedure is known (sorting, tax computation, password checking); learning it from examples would only add uncertainty. Early AI pushed this to expert domains with **expert systems** such as MYCIN: facts and rules extracted from specialists. The approach hit the **knowledge acquisition bottleneck**: extracting, expressing and maintaining the rules became the hard part.

Why rules fail for perception. Take handwritten digits. People recognise a “3” across changes of position, size, slant, stroke thickness, yet cannot write the procedure they use; rules about curves and endpoints meet endless exceptions. The lesson:

Having an ability $\neq$ having an explicit procedure that describes it.

When useful experience is available but the procedure is hard to specify, a second idea becomes attractive.

Second idea: make the machine learn. Apply the same picture to the machine: capacity to learn + experience $\rightarrow$ acquired ability. Designing the computational methods that make this possible *is* machine learning. It is still programming: the designer builds the learning system (what it can represent, what can change, what information guides the change, what counts as good); the task-specific behaviour is then acquired from data instead of being written as an exhaustive rule list.

1.2 Three forms of experience, three learning settings

Experience reaches a machine in three forms, each giving a different kind of information, and each leads to a setting:

Experience	What it contains	Setting
Examples	input + desired output	supervised learning
Observations	data with no supplied answer	unsupervised / self-supervised learning
Interactions	actions + their consequences (rewards)	reinforcement learning

Examples give direct guidance. Observations contain regularities that can be exploited without labels. In interaction the learner partly shapes its own future experience, because its actions change what happens next. These are settings, not kinds of systems; one system can use several.

1.3 Supervised learning in detail

Each training example pairs an input with a desired output, called the **target**, **label** or **ground truth**. Labels come from measurement (a temperature, a sale price), from human or expert annotation (an object in an image, a diagnosis), or from the future (did the customer repay?). “Ground truth” deserves care: measurements have errors, experts disagree, labels depend on conventions, and the quality of supervision bounds what can be learned.

The training cycle. The model takes an input, produces a prediction, the prediction is compared with the label, the error guides an update of the parameters, and the next example arrives. The computational process through which experience changes the model is called **training**.

Regression versus classification. The *output* decides the problem: numerical output (price, temperature, remaining lifetime) is **regression**; categorical output (digit identity, spam or not, species) is **classification**. The same image can give either, depending on what you ask (tumour size versus tumour type). Richer outputs exist: object detection (several objects and locations), semantic segmentation (a class per pixel).

Prediction, decision, action. A **prediction** is what the model says about the unknown output. A **decision** selects a response using the prediction and a criterion. An **action** is what is executed. A medical model predicts a disease probability; the decision is whether to order a test; the action is the test. They coincide in simple cases and separate exactly when different errors have different consequences (Lecture 2 makes this precise).

1.4 What is presented to the model: features and representations

The form of experience says what information exists; a separate question is what part is *presented* to the model. For tabular problems a designer picks **features** (area, location, age) and the model learns how they relate to the price. For perception (millions of pixels, a waveform, text) this is far harder, and for years systems relied on **feature engineering**: edges, corners, textures for vision; spectral features and mel-frequency cepstral coefficients for speech. The representation has a major effect on success.

Representation learning asks the machine to discover a useful representation itself; it predates deep learning but suits it well because successive layers build higher-level features from lower-level ones:

raw input $\rightarrow$ human-designed features $\rightarrow$ learned predictor
versus raw input $\rightarrow$ learned representation $\rightarrow$ learned predictor.

Humans still choose architecture, learning method, experience and objective; what changed is that more of the representation is acquired from data.

Self-supervision. A learning problem can be built from observations alone: predict a missing word from its context, a missing image region from the rest, a missing audio segment. The representation learned this way can later serve tasks with few labels (transfer). **From self-supervision to generation:** text supplies both context and what comes next, so an **autoregressive language model** trained to predict the next token is self-supervised; once trained, generation is just repeated prediction (predict, append, repeat). Two uses of one model: learned representations for downstream tasks, and learned predictive ability for generation.

1.5 Four ingredients, and the challenge of generalisation

The four ingredients of any learning system

Experience	what information is available?
Model	how can acquired knowledge be represented?
Learning algorithm	how does experience change the model?
Evaluation criterion	how is success judged?

The model determines what can be represented, the algorithm how experience changes it, the criterion which changes are useful, and the experience what evidence exists at all. Representation learning adds that the internal representation itself can be shaped by experience.

Experience is finite; deployment is not. **Generalisation** is the ability to perform well in situations not encountered during learning. A recogniser that is perfect on its own examples may fail on a new handwriting style, so performance on seen examples is only partial evidence:

$$\text{Finite experience} \xrightarrow{\text{learning}} \text{ability useful in new situations.}$$

Connection to information theory. Entropy measures uncertainty; information measures how much observing something reduces it. An input is useful when it carries information about the output. Two links to keep: **cross-entropy** measures how good predicted probabilities are and becomes the standard classification training criterion (Lecture 2); representation learning asks which information a representation should preserve.

The thread so far

Learning = experience changing a model so that it works on new situations. Supervised learning uses labelled examples; the output type decides regression versus classification; prediction, decision and action are distinct; the representation presented to the model matters and may itself be learned; everything is judged by generalisation, not by fit to the examples seen.

2 From representations to decisions (Lecture 2)

2.1 The input as a vector

The learner never receives the house or the image; it receives selected properties, encoded as numbers:

$$\text{object} \rightarrow \text{features} \rightarrow x = [x_1, \dots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d.$$

Components can be a normalised measurement in $[0, 1]$, a binary indicator in $\{0, 1\}$, or several binary components encoding one categorical feature (one-hot: neighbourhood A $\rightarrow (1, 0, 0)$). The representation fixes what information the learner can use: more relevant components help; irrelevant, redundant or unreliable ones raise d without adding information. Pixels and waveforms are numerical but do not encode shape or meaning explicitly, which is why representation learning matters.

2.2 Decision making when the distribution is known

Idealise: the input is a random vector X , the output a random variable Y , with joint distribution $P(X, Y)$. Once $X = x$ is seen, what remains is the **conditional distribution** $P(Y \mid X = x)$; houses with identical descriptions still sell at different prices, and this distribution carries that leftover uncertainty instead of forcing one output per input.

Knowing the probabilities does not say what to do; consequences do. If the true output is y and we choose action $a \in \mathcal{A}$, the **decision loss** $\ell_{\text{dec}}(y, a)$ is the cost. Keeping a separate from y matters because actions and outcomes need not live in the same set.

Bayes decision rule

$$f^*(x) \in \arg \min_{a \in \mathcal{A}} \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x].$$

For each input, compare the conditional expected loss (conditional risk) of every action and pick the smallest. “Bayes” here means optimal decision under a probability distribution; no prior on parameters is involved.

Regression. Under squared loss $\ell_{\text{dec}}(y, a) = (y - a)^2$ the rule gives the **conditional mean**:

$$f^*(x) = \mathbb{E}[Y \mid X = x].$$

Under absolute loss $|y - a|$ any **conditional median** is optimal; under an asymmetric loss (under- and over-estimation cost differently) a **conditional quantile**. So “the best prediction” has no meaning until the loss is specified: the loss decides which summary of $P(Y \mid x)$ to report.

Classification. With C classes the conditional distribution is the vector of posteriors $P(Y = c \mid X = x)$, $c = 1, \dots, C$. Prediction (the probabilities) and decision (a class) are visibly separate:

$$\{P(Y = c \mid x)\}_{c=1}^C \xrightarrow{\text{decision criterion}} f^*(x).$$

Bayes classifier under zero-one loss

If every error costs the same ($\ell_{01}(y, a) = \mathbb{I}(y \neq a)$), choose the most probable class:

$$f^*(x) = \arg \max_c P(Y = c \mid X = x).$$

Unequal costs: the threshold rule

Binary case, class 1 = the event to detect, $p(x) = P(Y = 1 \mid x)$, false positive cost C_{FP} , false negative cost C_{FN} , correct decisions free. Conditional expected costs:

$$\text{decide positive: } C_{FP}[1 - p(x)], \quad \text{decide negative: } C_{FN}p(x).$$

Choose the cheaper, i.e.

$$\text{decide positive} \iff p(x) > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Equal costs give the threshold 0.5. A costlier false negative (a missed attack, undetected fraud) *lowers* the threshold: less probability is needed to raise the alarm. A costlier false positive raises it.

The probabilities say what the input tells you; the loss says how to use it. The same $p(x)$ can lead to different decisions in different applications.

2.3 From the known distribution to finite data

In reality P is unknown. What we have is a finite **training set**

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N, \quad x_j^{(i)} = \text{feature } j \text{ of example } i,$$

modelled as independent, identically distributed (**i.i.d.**) draws from $P(X, Y)$. This assumption is a starting point; time series, spatial data and interacting agents violate independence, and the deployment distribution may differ. The central statistical step of supervised learning is: use the finite sample to build a rule that approximates the Bayes rule we would use if P were known.

2.4 First strategy: estimate locally with nearest neighbours

The Bayes rules need the conditional mean or the conditional class probabilities at x . Exact repeats of x are rare, so use the examples *near* x . Let $N_k(x)$ be the indices of the k training inputs closest to x .

k -nearest neighbours

$$\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)} \quad (\text{regression}), \quad \hat{P}_k(Y = c | x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbb{1}(y^{(i)} = c) \quad (\text{classification}).$$

Under zero-one loss the classification rule is the majority vote among the neighbours.

“Nearest” depends on the distance and the representation. The slides use the Minkowski family

$$d_p(x, z) = \left(\sum_{j=1}^d |x_j - z_j|^p \right)^{1/p}, \quad p = 1 \text{ Manhattan, } p = 2 \text{ Euclidean, Hamming for binary vectors,}$$

and a weighted version $d_w(x, z) = \sqrt{\sum_j w_j (x_j - z_j)^2}$. **Scaling matters:** income in dollars swamps age in years, so standardise each feature, $z_j = (x_j - \mu_j)/\sigma_j$. Irrelevant features make similar examples look far apart. KNN makes the role of the representation visible.

The hyperparameter k . Small k follows individual observations (sensitive to noise, jagged boundary); large k averages broadly (smoother, can erase real local structure). k is not estimated by the averaging rule; it is chosen during model development: a **hyperparameter**.

Why this exists

KNN is the most direct translation of the Bayes rule into data. Its weakness is what motivates everything after it.

The curse of dimensionality. KNN needs enough observations near the query. On $[0, 1]^d$ a grid with ten positions per axis has 10^d cells: 10^2 for $d = 2$, 10^{10} for $d = 10$. With a fixed dataset, points become sparse as d grows; a neighbourhood holding k points must cover a large region and the estimate loses its locality. Not every high-dimensional problem is hopeless, but local estimation cannot exploit high-dimensional inputs without extra structure. This motivates the second strategy: describe how predictions vary across the whole input space with a shared model and learn its adjustable quantities from all examples.

2.5 Second strategy: parameterised models

A **parameterised model** is a function $f_\theta(x)$ whose adjustable quantities θ are learned from data. The set it makes available is the **hypothesis space**

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}.$$

The model *form* decides how parameters are shared across inputs; θ picks one member. Choosing $\mathcal{H}$ introduces structure before any learning.

Regression models. Linear regression $f_\theta(x) = w^\top x + b$, $\theta = (w, b)$. A polynomial $f_\theta(x) = \theta_0 + \theta_1 x + \dots + \theta_p x^p$ shows the two levels: the coefficients are learned parameters, the degree p specifies the family.

Classification models. The model outputs an estimated probability vector

$$\hat{p}_\theta(x) = [\hat{P}_\theta(Y = 1 | x), \dots, \hat{P}_\theta(Y = C | x)]^\top \in \Delta^{C-1} = \left\{ p \in \mathbb{R}^C : p_c \geq 0, \sum_c p_c = 1 \right\},$$

the **probability simplex**. The decision rule (largest probability, or cost-weighted threshold) is applied afterwards.

Cross-entropy loss

For example $(x^{(i)}, y^{(i)})$ and predicted probabilities $\hat{p}_\theta(x^{(i)})$,

$$\ell_{CE}(y^{(i)}, \hat{p}_\theta(x^{(i)})) = -\log \hat{p}_{\theta, y^{(i)}}(x^{(i)}), \quad \hat{R}_{CE}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{\theta, y^{(i)}}(x^{(i)}).$$

Binary form with $\hat{p}$ = predicted probability of class 1: $\ell_{BCE}(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$.

Assigning 0.9, 0.5, 0.1 to the true class costs about 0.11, 0.69, 2.30: the penalty is graded and confident mistakes are punished hard. Zero-one loss cannot play this role: it is the same for a confident and a hesitant correct answer, changes only when the predicted class flips, and (slides) is piecewise constant with zero or undefined gradient, so nothing can be optimised. Cross-entropy is the smooth **surrogate**. Two properties make it the right surrogate: minimising it maximises the conditional likelihood of the labels (Lecture 4), and its expected value is minimised exactly when the predicted probabilities equal the true conditional probabilities (Appendix, next). A classifier trained with cross-entropy therefore estimates information that several decision rules can use.

Why cross-entropy learns probabilities (information theory)

For a true distribution p and an estimate $\hat{p}$ on the same alphabet,

$$H(p) = -\sum_c p_c \log p_c, \quad H(p, \hat{p}) = -\sum_c p_c \log \hat{p}_c, \quad D_{KL}(p \| \hat{p}) = \sum_c p_c \log \frac{p_c}{\hat{p}_c},$$

$$H(p, \hat{p}) = H(p) + D_{KL}(p \| \hat{p}).$$

$H(p)$ is fixed by the source and $D_{KL} \geq 0$, so expected cross-entropy is smallest when $\hat{p} = p$. Applied at every input,

$$R_{CE}(\theta) = H(Y | X) + \mathbb{E}_X [D_{KL}(p(X) \| \hat{p}_\theta(X))], \quad I(X; Y) = H(Y) - H(Y | X).$$

The learner shrinks the second term; the first, the uncertainty left after seeing X , belongs to the problem, so even an ideal classifier cannot go below $H(Y | X)$. Coding view: probability $\hat{p}_c$ is a code of $-\log_2 \hat{p}_c$ bits; the excess over $H_2(p)$ is D_{KL} ; a one-hot target makes the per-example cross-entropy $-\log \hat{p}_{\theta, y^{(i)}}$.

Parameters versus hyperparameters. Parameters are learned from training examples (polynomial coefficients, network weights). Hyperparameters are choices around that estimation (degree, k , number of layers, regularisation strength): some define the hypothesis space, others control the learning procedure.

model form and hyperparameters $\rightarrow \mathcal{H} \rightarrow \theta \rightarrow$ particular model.

2.6 Learning the parameters: empirical risk minimisation

Two losses must be told apart: ℓ_{train} guides estimation from data; ℓ_{dec} guides the action. They may coincide (squared loss for both) or differ (train with cross-entropy, decide with zero-one or application costs).

Population risk, empirical risk, ERM

$$R(\theta) = \mathbb{E}_{(X, Y) \sim P} [\ell_{\text{train}}(Y, f_\theta(X))], \quad \theta_{\mathcal{H}}^* \in \arg \min_{\theta \in \Theta} R(\theta),$$

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{\text{train}}(y^{(i)}, f_{\theta}(x^{(i)})), \quad \hat{\theta} \in \arg \min_{\theta \in \Theta} \hat{R}_{\text{train}}(\theta).$$

Population risk is what we care about (it averages over pairs never seen) but cannot be computed; the empirical risk replaces the expectation by the sample average, and choosing θ to minimise it is **empirical risk minimisation (ERM)**. $\theta_{\mathcal{H}}^*$ may be worse than the Bayes rule if $\mathcal{H}$ does not contain it. The hat on $\hat{\theta}$ records that it depends on the particular sample; another sample gives another $\hat{\theta}$.

Do not confuse

ERM is a good way to estimate *parameters* inside a fixed configuration, and a bad way to choose *complexity*. If $\mathcal{H}_1 \subseteq \mathcal{H}_2$, the best training fit in $\mathcal{H}_2$ can only be as good or better, and part of that improvement is fitting accidental variation. Raising the degree or lowering k keeps reducing training loss after performance on new data has started to degrade. Parameters need training data; hyperparameters need separate validation data.

Objective versus algorithm. The learning objective says which parameter values are preferred (ERM). The learning algorithm says how they are found: sometimes a closed form, usually iterative optimisation (later lectures). The full sequence separates three stages:

$$\mathcal{D} \xrightarrow[\text{learning}]{\ell_{\text{train}}} \text{learned model} \xrightarrow[\text{inference}]{x} \text{inferred quantity} \xrightarrow[\text{decision}]{\ell_{\text{dec}}} \text{action}.$$

2.7 Evaluating a learned model: three subsets

The population risk is unavailable, so held-out data estimate behaviour beyond the sample. The three subsets differ by what each is allowed to influence:

Training set $\mathcal{D}_{\text{train}}$	learn the parameters (through ℓ_{train})
Validation set $\mathcal{D}_{\text{val}}$	compare fitted configurations; select k , degree, representation, model family
Test set $\mathcal{D}_{\text{test}}$	one final estimate after everything is frozen

A subset stays informative only while its outputs are not used to make an earlier choice. Reusing validation results for many decisions adapts the development to that set, hence the third set; consulting the test set during development turns it into another validation set and makes its estimate optimistic.

Hyperparameter selection

For each candidate λ (a k , a degree, ...) learn parameters on training data, then compare on validation data:

$$\hat{\theta}_{\lambda} = \arg \min_{\theta} \hat{R}_{\text{train}}(\theta; \lambda), \quad \hat{\lambda} = \arg \min_{\lambda} \hat{R}_{\text{val}}(\hat{\theta}_{\lambda}; \lambda).$$

The validation criterion need not equal the training loss. **When data are limited: K -fold cross-validation.** Split the development data into K folds; for each configuration train K times, each time validating on one fold and training on the other $K - 1$, so every example validates once:

$$\hat{R}_{CV}(\lambda) = \frac{1}{K} \sum_{j=1}^K \hat{R}_{\text{val}}^{(j)}(\lambda).$$

After selecting $\hat{\lambda}$, refit on all development data; the test set stays untouched.

The thread so far

Known P + a loss $\Rightarrow$ Bayes rule (conditional mean; most probable class; cost threshold $C_{FP}/(C_{FP} + C_{FN})$). Unknown $P \Rightarrow$ finite i.i.d. sample. Estimate locally (KNN; killed by dimension) or through a parameterised family $\mathcal{H}$ fitted by ERM, with cross-entropy as the probability-learning loss. Training data fit θ , validation data choose hyperparameters, the test set is spent once.

3 From fitting to generalisation (Lecture 3)**3.1 Two risks and the generalisation gap**

ERM solves a *fitting* problem. The purpose of learning is performance on new cases, so compare two averages of the same loss for a fitted model $\hat{f}_{\mathcal{D}}$:

Empirical risk, population risk, generalisation gap

$$\hat{R}_{\mathcal{D}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, f(x^{(i)})), \quad R(f) = \mathbb{E}_{(X,Y) \sim P}[\ell(Y, f(X))], \quad G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}).$$

Both terms concern the same model; only the source of examples differs. A small gap means training performance transfers; a large positive gap means the training number flatters.

Why training performance is special. Training data are used twice: to select the model and then to evaluate it, $\mathcal{D} \rightarrow \hat{f}_{\mathcal{D}} \rightarrow \hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$, so the procedure can exploit accidental regularities of $\mathcal{D}$. Another sample $\mathcal{D}'$ would give another $\hat{f}_{\mathcal{D}'}$; this sampling variability is part of the problem. Low training loss is necessary evidence that optimisation worked and incomplete evidence that learning succeeded.

Stable versus spurious regularities. Cows photographed on grass, camels on sand: shape and background both predict every training label. Shape is the intended cue; background is a **shortcut** created by data collection. A cow on sand tells them apart. Training performance cannot, so dataset composition is part of the method; the same happens when a network segment predicts attacks or a scanner identifies a diagnosis. Vary nuisance factors when collecting; use **data augmentation** only with realistic, label-preserving transformations.

Inductive bias. A finite sample is compatible with many functions, including ones identical on the sample and different everywhere else. Learning therefore needs assumptions that prefer some solutions: the data and representation (which cues exist), the hypothesis space (which relationships are expressible), regularisation (which fits are favoured), and the optimiser (which minimiser it lands on). “Bias” here means preference, broader than the statistical bias below. Generalisation succeeds when these assumptions stay valid at deployment.

3.2 Model capacity, underfitting, overfitting

Capacity is how varied the functions in $\mathcal{H}$ can be. It is not the parameter count alone: structure, constraints, regularisation, optimisation and the data’s amount and geometry all enter. Capacity-controlling hyperparameters: polynomial degree, tree depth, number of units, and k in KNN (where *smaller* k means more capacity). For nested families $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \dots$,

$$\min_{f \in \mathcal{H}_{m+1}} \hat{R}_{\text{train}}(f) \leq \min_{f \in \mathcal{H}_m} \hat{R}_{\text{train}}(f),$$

so training error alone always favours the richer family; population risk need not follow. **Underfitting:** too restrictive, high training and validation error (a line through curved data). **Overfitting:**

too sensitive to the particular sample, low training error and much higher validation error (a high-degree polynomial oscillating between points). The question is not whether the model is complicated in isolation but whether its effective flexibility fits the available evidence; a family unstable on a small sample can be well constrained by a large one. The classical **capacity curve**: training error keeps falling, validation error falls then rises.

3.3 Bias and variance

Imagine redrawing training sets $\mathcal{D}_1, \mathcal{D}_2, \dots$ from the same population and running the same procedure; the fitted functions differ. For squared-error regression let $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$, the average fitted prediction.

Bias, variance, and the decomposition

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \quad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2].$$

With $Y = f^*(X) + \varepsilon$, $\mathbb{E}[\varepsilon \mid X] = 0$, $\text{Var}(\varepsilon \mid X = x) = \sigma^2(x)$, adding and subtracting $\bar{f}(x)$ inside the squared error and taking expectations over noise and sample (the cross terms vanish) gives

$$\mathbb{E}_{\mathcal{D}, Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \underbrace{\sigma^2(x)}_{\text{irreducible noise}} + \underbrace{\text{Bias}(x)^2}_{\text{systematic error}} + \underbrace{\text{Var}(x)}_{\text{sample sensitivity}}.$$

Bias: would the *average* fit miss the target in a consistent direction? Restrictive models and poor features cause it, and more data does not remove it if the family cannot express the relationship. **Variance**: does swapping the sample change the prediction at the same input? High capacity, small data, irrelevant variables and unstable feature selection raise it. Raising capacity typically lowers bias and raises variance, producing the U-shaped validation curve; this is a diagnostic principle, not a law.

Beyond the U. Overparameterised models can show **double descent**: validation error spikes near the **interpolation threshold** (the smallest capacity that fits the sample exactly) and may fall again beyond it, depending on regularisation, optimiser, data geometry and which interpolating solution is chosen. The same can happen over *training time* at fixed size. So examine validation behaviour rather than assuming the U. Three sources of excess error: **approximation error** ($\mathcal{H}$ cannot express a good function), **estimation error** (finite data cannot identify the best member), **optimisation error** (the algorithm fails to reach a good solution). More capacity trades approximation error for estimation error.

3.4 Controlling complexity

Options: restrict the family (depth, degree), raise k , select features, penalise parameters, stop early, collect more relevant data. Each encodes an assumption about which fits generalise.

Regularised ERM

$$\hat{\theta}_{\lambda} = \arg \min_{\theta \in \Theta} [\hat{R}_{\text{train}}(\theta) + \lambda \Omega(\theta)], \quad \Omega_2(\theta) = \|\theta\|_2^2 = \sum_j \theta_j^2, \quad \Omega_1(\theta) = \|\theta\|_1 = \sum_j |\theta_j|.$$

Equivalent view: constrained sets $\Theta_c = \{\theta : \Omega(\theta) \leq c\}$ give nested families $\mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$ for $c_1 < c_2$; a stronger penalty confines the fit to a smaller effective family without changing the parameter count. λ is a hyperparameter chosen on validation data: $\hat{\lambda} \in \arg \min_{\lambda} \hat{R}_{\text{val}}(\hat{f}_{\lambda})$.

L_2 (ridge) shrinks coefficients smoothly and keeps all variables. L_1 (lasso) has a constraint region (a diamond in 2-D) whose corners lie on the axes, so the optimum often sits at a corner with some coefficients exactly zero: embedded feature selection, though correlated features make the selected

subset unstable.

Early stopping. Iterative optimisation produces $\hat{\theta}_0, \hat{\theta}_1, \dots$; early iterations learn recurring patterns, late ones fit exceptions and noise. Keep the checkpoint with the smallest validation risk, $t^* = \arg \min_t \hat{R}_{\text{val}}(\hat{\theta}_t)$, $\hat{\theta}_{ES} = \hat{\theta}_{t^*}$: training time controls effective capacity at fixed parameter count. Validation loss is noisy, so a **patience** p lets training continue p epochs without improvement before stopping while retaining the best checkpoint; too short stops on a fluctuation or misses an epoch-wise second descent.

3.5 Feature selection and model selection

Feature selection changes the representation: it removes irrelevant variables (which raise estimation difficulty), reduces redundancy, aids interpretation, and drops variables unavailable at decision time. It is a data-dependent choice and must be validated like any other. Exhaustive search over 2^d subsets, $S^* \in \arg \min_S \hat{R}_{\text{val}}(\hat{f}_S)$, is infeasible ($d = 30$ exceeds a billion). Alternatives: a **correlation filter** $s_j = |\text{Corr}(X_j, Y)|$ (fast, model-free, but blind to nonlinearity, interactions and redundancy; weak marginal correlation does not prove uselessness); **forward selection** (add the feature giving the best validation gain) and **backward elimination** (remove the least useful), both greedy. Every data-dependent scoring belongs inside the training pipeline and, in cross-validation, must be redone inside each training fold, otherwise validation information leaks into the representation.

Generalising from λ to a whole configuration h (features, hyperparameters, thresholds): $\hat{h} \in \arg \min_{h \in \mathcal{C}} \hat{R}_{\text{val}}(\hat{f}_h)$, and in K -fold form

$$\hat{R}_{CV}(h) = \frac{1}{K} \sum_{j=1}^K \hat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)}), \quad \hat{h} \in \arg \min_h \hat{R}_{CV}(h).$$

Each observation validates once and trains $K - 1$ times; cost about K fits per configuration; random folds assume exchangeable observations. **Validation can be overused:** the minimum of many noisy estimates is optimistic; the untouched test set protects against this and cannot repair it if its results feed back into development.

3.6 Reliable evaluation: splits, metrics, leakage

The split must reproduce the deployment novelty. Random split: only when new cases are like independent draws from the same population. **Stratified:** random but preserving class proportions (stabilises rare-class counts; cannot override dependence). **Grouped:** every device, user, patient or organisation entirely in one subset, so performance is measured on unseen groups. **Temporal:** train on the past, evaluate on later data; reveals drift.

Metrics for imbalanced classes (positive = attack)

$$\begin{aligned} \text{Recall} &= \frac{TP}{TP + FN}, & \text{Precision} &= \frac{TP}{TP + FP}, & \text{Specificity} &= \frac{TN}{TN + FP}, \\ \text{BalancedAcc} &= \frac{\text{Recall} + \text{Specificity}}{2}, & F_1 &= \frac{2 \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}. \end{aligned}$$

With 1% malicious traffic, “always benign” scores 99% accuracy and detects nothing. F_1 is high only when both precision and recall are, but ignores true negatives and weighs the two symmetrically. No metric is universally right; metric and threshold must follow the costs of missed attacks, false alarms and alert volume.

Leakage: development uses information unavailable at deployment-time decisions. Forms: duplicates across subsets; the same device, session or patient in training and test; predictors created

after the outcome or after an investigation; preprocessing fitted on validation or test data; hyperparameters tuned on test results. It inflates the estimate. Preprocessing is part of the fitted pipeline: standardisation estimates

$$\hat{\mu}_j = \frac{1}{N_{tr}} \sum_{i \in tr} x_j^{(i)}, \quad \hat{\sigma}_j^2 = \frac{1}{N_{tr}} \sum_{i \in tr} (x_j^{(i)} - \hat{\mu}_j)^2$$

on the training set only and reuses them unchanged on validation, test and deployment data; likewise imputation, feature selection, dimensionality reduction and resampling, per fold in cross-validation.

Diagnosis with learning curves (error versus training-set size, training and validation):

Pattern	Likely issue	Response
both errors high	underfitting	better representation or more capacity
training low, validation high	high variance	regularise, reduce capacity, more representative data
errors close, both falling with N	data-limited	more representative examples will help
test degrades over time	distribution shift	refresh data, time-aware evaluation

The thread so far

Training loss is a fit, not a generalisation estimate. Inductive bias (data, representation, $\mathcal{H}$, regularisation, optimiser) is unavoidable. Capacity trades approximation for estimation error; bias is systematic across samples, variance is spread across samples, noise is irreducible. Control capacity by family, penalty (L_2 shrinks, L_1 zeroes), early stopping, feature selection. Validation chooses, test measures once, the split mirrors deployment, nothing leaks.

4 Discriminative and generative classification (Lecture 4)

4.1 The decision is easy; the posterior is the problem

With $Y \in \{1, \dots, C\}$ and $X \in \mathbb{R}^d$, if the posterior $p(y = c | x)$ were known the Bayes rule $f^*(x) \in \arg \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) | X = x]$ would decide, and under zero-one loss $f^*(x) = \arg \max_c p(y = c | x)$. The learning problem is that the posterior must be estimated from $\mathcal{D}_N$. Two routes:

- **Discriminative:** approximate the posterior directly with $p_\theta(y = c | x)$; the fit is $p_{\hat{\theta}}(y = c | x)$. It asks *where the label changes* as a function of the features.
- **Generative:** model the **prior** $p(y = c)$ and the **class-conditional** $p(x | y = c)$, then obtain the posterior with Bayes' rule. It builds a probabilistic description of each class and compares how well each explains the input.

Neither is universally better; it depends on data amount, credibility of assumptions, representation and objective. “Generative” names the modelled direction, not the task: a class-conditional model can classify an input or, in principle, generate inputs of a chosen class, which links these classifiers to generative AI. Naive Bayes remains a cheap baseline for spam; class-conditional models score how compatible an event is with normal or known-malicious behaviour.

4.2 The discriminative route

Scores $\rightarrow$ probabilities $\rightarrow$ action. The model first outputs a real-valued **score** (logit) per class, then a probability, then a decision: $\{s_{\hat{\theta},c}(x)\} \rightarrow \{p_{\hat{\theta}}(y = c | x)\} \rightarrow \hat{y}(x)$. The score family fixes the geometry; the probability transform quantifies relative uncertainty; the decision loss fixes the action.

Sigmoid and softmax

Binary: one score, $p_{\hat{\theta}}(y = 1 | x) = \sigma(s_{\hat{\theta}}(x)) = \frac{1}{1 + e^{-s_{\hat{\theta}}(x)}}$, and under zero-one loss

$$\hat{y}(x) = 1 \iff p_{\hat{\theta}}(y = 1 | x) > \frac{1}{2} \iff s_{\hat{\theta}}(x) > 0.$$

Multiclass: one score per class, $p_{\theta}(y = c | x) = \frac{\exp s_{\theta,c}(x)}{\sum_{k=1}^C \exp s_{\theta,k}(x)}$. Adding a constant to all scores changes nothing, so only $C - 1$ contrasts are identifiable; C scores are kept for symmetry and a simple output layer.

Logistic regression uses $s_{\hat{\theta}}(x) = \hat{w}^{\top}x + \hat{b}$, so the 0.5 contour is the hyperplane $\hat{w}^{\top}x + \hat{b} = 0$. A nonlinear score $g(x)$ gives the boundary $g(x) = 0$: the threshold does not make geometry linear, the score family does. Cost-sensitive threshold again: with $r(x) = p(y = 1 | x)$, $\hat{y} = 1 \iff r(x) > C_{FP}/(C_{FP} + C_{FN})$; this encodes the operational tradeoff and is distinct from correcting for imbalance or prevalence.

Cross-entropy = negative conditional log-likelihood

$$\ell_{CE}^{(i)}(\theta) = - \sum_{c=1}^C \mathbb{I}(y^{(i)} = c) \log p_{\theta}(y = c | x^{(i)}) = - \log p_{\theta}(y^{(i)} | x^{(i)}),$$

$$L_{\text{cond}}(\theta) = \prod_{i=1}^N p_{\theta}(y^{(i)} | x^{(i)}),$$

and $-\log L_{\text{cond}}(\theta) = -\sum_i \log p_{\theta}(y^{(i)} | x^{(i)}) = N \cdot \hat{R}_{CE}(\theta)$ (labels conditionally independent given inputs). Minimising cross-entropy is maximising the conditional likelihood.

Deep discriminative classifiers learn a chain of representations and end in scores:

$$h^{(0)} = x, \quad h^{(\ell)} = \varphi(W_{\ell}h^{(\ell-1)} + b_{\ell}), \quad s_{\hat{\theta}}(x) = W_L h^{(L-1)} + b_L, \quad p_{\hat{\theta}}(y = c | x) = \text{softmax}(s_{\hat{\theta}}(x))_c,$$

trained end to end with cross-entropy; the representation is optimised for the label, not for explaining the input.

What the discriminative objective does not give. It need not learn whether x is plausible; a large maximum posterior does not mean the input resembles training data (softmax is a *relative* comparison among known classes, so an unfamiliar input can still get a posterior near one); it gives no mechanism to sample inputs or marginalise missing features. **Class imbalance** is a separate issue: the fitted posterior reflects training frequencies, and the real difficulty is too few minority examples to estimate the boundary; weighting or resampling helps in that regime and may otherwise distort calibration.

4.3 The generative route**Generative story, joint, posterior**

$Y \sim \text{Categorical}(\pi)$, then $X | (Y = c) \sim p_{\theta}(\cdot | y = c)$; parameters $\phi = (\pi, \theta)$, $\pi_c = p(y = c)$.

$$p_{\phi}(x, y = c) = \pi_c p_{\theta}(x | y = c), \quad p_{\phi}(y = c | x) = \frac{\pi_c p_{\theta}(x | y = c)}{\sum_{k=1}^C \pi_k p_{\theta}(x | y = k)}.$$

Classification compares prior-weighted likelihoods; a rare class wins if it explains the input much better. Example: benign $\pi = 0.99$, $p(x | \text{benign}) = 0.001$; attack $\pi = 0.01$, $p(x | \text{attack}) = 0.20$. Products 0.00099 and 0.002, so $p(\text{attack} | x) = 0.002/(0.002 + 0.00099) \approx 0.67$.

Generative maximum likelihood

$$L(\phi) = \prod_{i=1}^N p_{\phi}(x^{(i)}, y^{(i)}), \quad -\log L(\phi) = -\sum_{i=1}^N \left[\log \pi_{y^{(i)}} + \log p_{\theta}(x^{(i)} | y^{(i)}) \right].$$

Each example must be explained in both its class and its features, a stronger demand than explaining the class given the features. Two separate approximations: **shape approximation** (the family $p_{\theta}(x | y)$ may not contain the truth) and **finite-sample estimation** ($\hat{\phi}$ differs from the best member). Both move the fitted posterior away from the true one.

4.4 Gaussian class models: LDA and QDA

For continuous features, $p_{\theta}(x | y = c) = \mathcal{N}(x; \mu_c, \Sigma_c)$: μ_c sets the centre, Σ_c the scale, orientation and dependence (spherical gives circular contours, correlation rotates and elongates them).

Closed-form maximum likelihood

With N_c examples in class c ,

$$\hat{\pi}_c = \frac{N_c}{N}, \quad \hat{\mu}_c = \frac{1}{N_c} \sum_{i: y^{(i)}=c} x^{(i)}, \quad \hat{\Sigma}_c = \frac{1}{N_c} \sum_{i: y^{(i)}=c} (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^{\top}.$$

The denominator is N_c (ML), not the $N_c - 1$ of the unbiased sample covariance. No iteration needed.

One-dimensional check: class 0 = {0, 1, 2, 0, 1, 2}, class 1 = {3, 4, 5} give $\hat{\pi} = (2/3, 1/3)$, $\hat{\mu} = (1, 4)$, $\hat{\sigma}_0^2 = \hat{\sigma}_1^2 = 2/3$; at $x = 2$ comparing $\hat{\pi}_0 \mathcal{N}(2; 1, 2/3)$ and $\hat{\pi}_1 \mathcal{N}(2; 4, 2/3)$ gives $p(y = 0 | 2) \approx 0.950$: both distance to the centre and the larger prior count.

Generative class scores, LDA, QDA

Define $s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \log p_{\hat{\theta}}(x | y = c)$; then $\log p_{\hat{\phi}}(y = c | x) = s_{\hat{\phi},c}(x) - \log \sum_k \exp s_{\hat{\phi},k}(x)$, and the second term is common to all classes, so decide on the scores.

LDA (shared covariance Σ): dropping terms common to all classes,

$$s_{\hat{\phi},c}(x) = x^{\top} \hat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^{\top} \hat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c,$$

linear in x , so equating two scores gives a *linear* boundary (“linear” names the boundary, not the density).

QDA (class-specific Σ_c):

$$s_{\hat{\phi},c}(x) = -\frac{1}{2}(x - \hat{\mu}_c)^{\top} \hat{\Sigma}_c^{-1} (x - \hat{\mu}_c) - \frac{1}{2} \log |\hat{\Sigma}_c| + \log \hat{\pi}_c,$$

a squared Mahalanobis distance to the centre plus a term for how diffuse the class is; the quadratic terms give curved boundaries.

Principle: assumptions about the class-conditionals determine the geometry of the boundary. LDA shares structure and estimates fewer parameters; QDA is more flexible and needs more data. A $d \times d$ covariance has $d(d+1)/2$ parameters, 5050 per class at $d = 100$; with few observations it is unstable or singular. **Covariance shrinkage** $\hat{\Sigma}_{\lambda} = (1 - \lambda)\hat{\Sigma} + \lambda\alpha I$, $0 \leq \lambda \leq 1$, chosen on validation, interpolates between the sample covariance and a scaled identity (no correlations, one reference variance). Pushing this simplification further gives Naive Bayes.

4.5 Naive Bayes

Conditional independence and the additive score

$$p_{\theta}(x \mid y = c) = \prod_{j=1}^d p_{\theta}(x_j \mid y = c), \quad s_{\hat{\phi}, c}(x) = \log \hat{\pi}_c + \sum_{j=1}^d \log p_{\hat{\theta}}(x_j \mid y = c).$$

Each feature is a separate clue once the class is known. The assumption is strong (flow duration, packet counts and byte statistics stay correlated within an attack class) but it turns one hard d -dimensional density estimate into d one-dimensional ones, which are stable with little data. A misspecified density can still rank classes correctly: exact densities are sufficient, not necessary, for the right argmax. The risk: correlated features repeat the same evidence, so the classifier may classify well while giving probabilities that are too extreme; removing redundant features improves calibration. Each one-feature model can be anything that fits the feature's support and shape (Gamma or log-normal for positive skewed, Student t for heavy tails, Poisson or negative binomial for counts, histograms or kernel density estimates with enough data), validated like any choice; $\log(1+x)$ often makes a skewed positive feature near-Gaussian, while centring and scaling do not change its shape.

Variants. Gaussian (continuous measurements, flow statistics), Bernoulli (binary presence indicators), Multinomial (token or event counts, documents), Hybrid (a distribution per feature type for mixed tabular data; contributions multiply under one class prior, which appears only once).

Gaussian NB and variance smoothing. $x_j \mid (y = c) \sim \mathcal{N}(\mu_{cj}, \sigma_{cj}^2)$. A zero fitted variance makes the density undefined, a tiny one lets a single feature dominate, so regularise $\hat{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$ with s_j^2 the overall training variance of feature j and a small $\lambda > 0$ (validated if it matters).

Multinomial Naive Bayes for text and additive smoothing

Vocabulary of size V , document = count vector $x = (x_1, \dots, x_V)$, $\theta_{cj} = p_{\theta}(\text{token} = j \mid y = c)$, $\sum_j \theta_{cj} = 1$. Dropping the multinomial coefficient (common to all classes),

$$p_{\theta}(x \mid y = c) \propto \prod_{j=1}^V \theta_{cj}^{x_j} \quad (0^0 = 1; \text{ counts kept, order ignored}).$$

With n_{cj} the training count of token j in class c , additive smoothing with $\alpha > 0$:

$$\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_{\ell=1}^V n_{c\ell} + \alpha V}, \quad \text{categorical feature with } K_j \text{ levels: } p_{\hat{\theta}}(x_j = v \mid y = c) = \frac{n_{c j v} + \alpha}{n_{cj} + \alpha K_j}.$$

Why smoothing: vocabulary (free, win, prize, meeting, project, report), spam counts (3, 2, 2, 0, 0, 0), ham counts (0, 0, 0, 3, 2, 2), seven tokens each. Unsmoothed, “free meeting” has likelihood $\frac{3}{7} \cdot \frac{0}{7} = 0$ under spam and $\frac{0}{7} \cdot \frac{3}{7} = 0$ under ham: one unseen word zeroes the product and no posterior exists. With $\alpha = 1$, $V = 6$: $\hat{\theta}_{\text{spam}} = \frac{1}{13}(4, 3, 3, 1, 1, 1)$, $\hat{\theta}_{\text{ham}} = \frac{1}{13}(1, 1, 1, 4, 3, 3)$; equal priors give unnormalised scores $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13} = \frac{2}{169}$ for both, so $p(\text{spam} \mid x) = \frac{1}{2}$: the evidence balances exactly. Larger α moves estimates toward uniform. Unseen categorical values need a representation: create an “other” level during training and map unseen values to it.

4.6 Comparing the two routes

	Discriminative	Generative
Modelled quantity	posterior $p_\theta(y x)$ or boundary	prior and class model $\pi_c p_\theta(x y = c)$
Input assumptions	usually fewer	usually stronger
Primary objective	predict the class	describe class-specific inputs
Missing features	separate treatment	can sometimes be marginalised
Sampling inputs	usually unavailable	possible in principle
Raw high-dimensional input	often practical	full density estimation is hard

Generative structure shares a restricted description of each class and so can reduce estimation error with little labelled data when the assumptions roughly hold; a wrong family creates approximation bias that more data does not remove. A flexible discriminative model can eventually reach lower error because it spends no capacity on aspects of X irrelevant to Y . There is no universal learning curve; the question is which assumptions are credible and which objective matters.

4.7 Imbalance, prior shift, calibration

Imbalance without shift. If training and deployment share the joint distribution, an imbalanced prior is not an error; the posterior should reflect it. The real problem is few minority examples (small N_1), where weighting or resampling can help estimation; otherwise they may distort probabilities. A classifier is **calibrated** if among cases given probability u about a fraction u are positive. Calibration concerns the meaning of probabilities; the cost threshold concerns the action. Separate questions.

Class-prior shift and its correction

Shift: $p_d(x | y = c) = p_t(x | y = c)$ but $\pi_{d,c} \neq \pi_{t,c}$ (t = training, d = deployment; e.g. 20% attacks in training, 5% in deployment).

Generative: keep the fitted class-conditionals, swap the priors: $p_d(y = c | x) = \frac{\pi_{d,c} p_{\hat{\theta}}(x | y = c)}{\sum_k \pi_{d,k} p_{\hat{\theta}}(x | y = k)}$.

Binary discriminative, with $r_t(x) = p_t(y = 1 | x)$: the likelihood ratio is unchanged, so posterior odds scale by the prior-odds ratio, $\frac{r_d}{1 - r_d} = \frac{\pi_d / (1 - \pi_d)}{\pi_t / (1 - \pi_t)} \cdot \frac{r_t}{1 - r_t}$, i.e.

$$r_d(x) = \frac{(\pi_d / \pi_t) r_t(x)}{(\pi_d / \pi_t) r_t(x) + \frac{1 - \pi_d}{1 - \pi_t} [1 - r_t(x)]}.$$

This needs r_t to be calibrated. The generative route makes the prior an explicit component; the discriminative route is corrected after the fact through the quality of its probabilities.

4.8 Unfamiliar inputs and combined systems

A high $\max_c p_{\hat{\theta}}(y = c | x)$ only says one known class outranks the others; it does not say x resembles training data, which matters when a new attack mechanism lies far from every known class. A generative model adds a compatibility check: pick $c^* = \arg \max_c p_{\hat{\theta}}(y = c | x)$, then

$$\hat{y} = \begin{cases} c^* & \text{if } \log p_{\hat{\theta}}(x | y = c^*) \geq \tau_{c^*}, \\ \text{reject} & \text{otherwise,} \end{cases}$$

with τ_c validated on data that reproduce the intended novelty (attack types held out of training). Low likelihood is evidence of incompatibility, not a guarantee; high-dimensional likelihoods behave oddly and a misspecified model can reject valid cases. Modern systems combine the routes: a

discriminative model ranks known classes, a generative or anomaly model checks compatibility, and the policy accepts, sends to review, or rejects.

Language models. An autoregressive model is trained generatively, $p_\theta(w_{1:T}) = \prod_{t=1}^T p_\theta(w_t | w_{<t})$ (next-token prediction). A sequence-classification head instead models $p_\psi(y | w_{1:T})$ with the discriminative objective $L_{\text{disc}}(\psi) = -\sum_i \log p_\psi(y^{(i)} | w_{1:T_i}^{(i)})$, one label per sequence; or a prompt asks the model to generate or score label tokens. Generative pretraining, discriminative downstream objective.

Choosing a model. Can a credible $p_\theta(x | y)$ be stated? Then its structure may save labelled data. Is the goal accurate prediction with many labels? A flexible discriminative model avoids needless input assumptions. Is the raw input complex and high-dimensional? Learn a representation first, then a simple head. Labels like “generative” never replace deployment-like validation on the same fixed split and rows for every model.

The thread so far

Posterior + decision loss $\Rightarrow$ decision. Discriminative: scores $\rightarrow$ sigmoid/softmax $\rightarrow$ cross-entropy = negative conditional log-likelihood; boundary geometry comes from the score family. Generative: prior $\times$ class-conditional, Bayes’ rule, joint likelihood; Gaussian models give LDA (linear) and QDA (quadratic); Naive Bayes factorises features and needs variance smoothing or additive smoothing. Imbalance is an estimation problem, prior shift is a posterior correction, calibration is about probability meaning, and confidence is not familiarity.

One-glance sheet: every formula

Bayes decision rule	$f^*(x) \in \arg \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$
Squared / absolute / asymmetric loss	conditional mean $\mathbb{E}[Y \mid x]$ / conditional median / conditional quantile
Zero-one loss	$f^*(x) = \arg \max_c P(Y = c \mid x)$
Cost threshold	positive iff $p(x) > C_{FP}/(C_{FP} + C_{FN})$
KNN	$\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}$; $\hat{P}_k(c \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbb{1}(y^{(i)} = c)$
Minkowski distance	$d_p(x, z) = (\sum_j x_j - z_j ^p)^{1/p}$; standardise $z_j = (x_j - \mu_j)/\sigma_j$
Curse of dimensionality	10^d grid cells for ten positions per axis
Hypothesis space	$\mathcal{H} = \{f_\theta : \theta \in \Theta\}$; linear $w^\top x + b$; polynomial $\sum_{k \leq p} \theta_k x^k$
Simplex	$\Delta^{C-1} = \{p \geq 0, \sum_c p_c = 1\}$
Cross-entropy	$\ell_{CE} = -\log \hat{p}_{\theta, y}(x)$; $\hat{R}_{CE} = -\frac{1}{N} \sum_i \log \hat{p}_{\theta, y^{(i)}}(x^{(i)})$; BCE $= -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$
Information theory	$H(p, \hat{p}) = H(p) + D_{KL}(p \parallel \hat{p})$; $R_{CE} = H(Y \mid X) + \mathbb{E}_X D_{KL}$; $I(X; Y) = H(Y) - H(Y \mid X)$
Risks, ERM	$R(\theta) = \mathbb{E}[\ell(Y, f_\theta(X))]$; $\hat{R}_{\text{train}} = \frac{1}{N} \sum_i \ell(y^{(i)}, f_\theta(x^{(i)}))$; $\hat{\theta} \in \arg \min \hat{R}_{\text{train}}$
Selection, CV	$\hat{\theta}_\lambda = \arg \min_\theta \hat{R}_{\text{train}}(\theta; \lambda)$; $\hat{\lambda} = \arg \min_\lambda \hat{R}_{\text{val}}(\hat{\theta}_\lambda)$; $\hat{R}_{CV} = \frac{1}{K} \sum_j \hat{R}_{\text{val}}^{(j)}$
Generalisation gap	$G(\hat{f}) = R(\hat{f}) - \hat{R}_{\mathcal{D}}(\hat{f})$
Nested capacity	$\min_{\mathcal{H}_{m+1}} \hat{R}_{\text{train}} \leq \min_{\mathcal{H}_m} \hat{R}_{\text{train}}$
Bias-variance	$\mathbb{E}[(Y - \hat{f}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x)$, $\text{Bias} = \bar{f} - f^*$, $\text{Var} = \mathbb{E}_{\mathcal{D}}[(\hat{f} - \bar{f})^2]$
Regularised ERM	$\arg \min_\theta [\hat{R}_{\text{train}} + \lambda \Omega(\theta)]$; $\Omega_2 = \ \theta\ _2^2$ (shrink), $\Omega_1 = \ \theta\ _1$ (sparsify)
Early stopping	$t^* = \arg \min_t \hat{R}_{\text{val}}(\hat{\theta}_t)$ with patience p
Filter, subsets	$s_j = \text{Corr}(X_j, Y) $; 2^d subsets; forward / backward greedy
Metrics	Recall $\frac{TP}{TP+FN}$, Precision $\frac{TP}{TP+FP}$, Specificity $\frac{TN}{TN+FP}$, BalAcc, $F_1 = \frac{2PR}{P+R}$
Standardisation (train only)	$\hat{\mu}_j = \frac{1}{N_{tr}} \sum x_j^{(i)}$, $\hat{\sigma}_j^2 = \frac{1}{N_{tr}} \sum (x_j^{(i)} - \hat{\mu}_j)^2$
Sigmoid, softmax	$\sigma(s) = 1/(1 + e^{-s})$, decide 1 iff $s > 0$; $\text{softmax}_c = e^{s_c} / \sum_k e^{s_k}$
Conditional likelihood	$L_{\text{cond}} = \prod_i p_\theta(y^{(i)} \mid x^{(i)})$; $-\log L_{\text{cond}} = N \hat{R}_{CE}$
Network	$h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$, $s = W_L h^{(L-1)} + b_L$
Generative model	$p_\phi(x, c) = \pi_c p_\theta(x \mid c)$; $p(c \mid x) = \pi_c p(x \mid c) / \sum_k \pi_k p(x \mid k)$; $-\log L = -\sum_i [\log \pi_{y^{(i)}} + \log p_\theta(x^{(i)} \mid y^{(i)})]$
Gaussian ML	$\hat{\pi}_c = N_c/N$, $\hat{\mu}_c = \text{class mean}$, $\hat{\Sigma}_c = \frac{1}{N_c} \sum (x - \hat{\mu}_c)(x - \hat{\mu}_c)^\top$
Scores	$s_c = \log \hat{\pi}_c + \log p(x \mid c)$; $\log p(c \mid x) = s_c - \log \sum_k e^{s_k}$
LDA	$s_c = x^\top \hat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^\top \hat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c$ (linear boundary)
QDA	$s_c = -\frac{1}{2} (x - \hat{\mu}_c)^\top \hat{\Sigma}_c^{-1} (x - \hat{\mu}_c) - \frac{1}{2} \log \hat{\Sigma}_c + \log \hat{\pi}_c$ (quadratic)
Covariance size, shrinkage	$d(d+1)/2$ parameters; $\hat{\Sigma}_\lambda = (1 - \lambda) \hat{\Sigma} + \lambda \alpha I$
Naive Bayes	$p(x \mid c) = \prod_j p(x_j \mid c)$; $s_c = \log \hat{\pi}_c + \sum_j \log p(x_j \mid c)$; $\hat{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$
Multinomial NB	$p(x \mid c) \propto \prod_j \theta_{cj}^{x_j}$; $\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_\ell n_{c\ell} + \alpha V}$; categorical $\frac{n_{cj} + \alpha K_j}{n_{cj} + \alpha K_j}$
Prior shift	generative: swap π ; discriminative: $r_d = \frac{(\pi_d/\pi_t)r_t}{(\pi_d/\pi_t)r_t + \frac{1-\pi_d}{1-\pi_t}(1-r_t)}$
Reject option	$\hat{y} = c^*$ if $\log p(x \mid c^*) \geq \tau_{c^*}$, else reject
Language models	$p(w_{1:T}) = \prod_t p(w_t \mid w_{<t})$; head $L_{\text{disc}} = -\sum_i \log p_\psi(y^{(i)} \mid w_{1:T_i}^{(i)})$

One-glance sheet: the distinctions

Prediction / decision / action	what the model says / what response is chosen under a criterion / what is executed
Regression / classification	numerical output / categorical output; decided by the output, not the input
Supervised / unsupervised / self-sup. / RL	examples / observations / observations with a constructed signal / interactions
ℓ_{train} / ℓ_{dec}	guides estimation / guides the action; may coincide or differ
Parameter / hyperparameter	learned from training data / chosen on validation data
Objective / algorithm	which values are preferred / how they are found
Train / validation / test	fit θ / select h / measure once, after freezing
Population / empirical risk	expectation over P (unknown) / sample average (observable, sample-dependent)
Inductive bias / statistical bias	preference among solutions / $\bar{f} - f^*$, systematic error across samples
Bias / variance / noise	average fit misses / fits spread across samples / $\sigma^2(x)$, irreducible
Underfitting / overfitting	both errors high / training low, validation high
Approximation / estimation / optimisation error	$\mathcal{H}$ too poor / data cannot pick the best member / solver fails
L_2 / L_1	shrink all / zero some (corners of the diamond)
Random / stratified / grouped / temporal split	i.i.d. cases / keep class ratios / unseen groups / future data
Discriminative / generative	$p(y x)$ directly / $p(y)$, $p(x y)$ then Bayes
Shape approximation / finite-sample estimation	wrong family / wrong member of the family
LDA / QDA	shared Σ , linear boundary, fewer parameters / own Σ_c , curved boundary, more data
Imbalance / prior shift	estimation problem (few minority examples) / posterior correction (priors changed)
Calibration / threshold	meaning of probabilities / action taken from them
Confidence / familiarity	max posterior among known classes / compatibility with the training data
